

Ahhh yes — I see exactly where the confusion is. 🤔 You're comparing this with your **React + Parcel** experience, which is actually the best way to understand it.

Let's put them **side by side**, starting from the browser typing the URL.

React + Parcel vs Next.js

■ React + Parcel

Suppose your project is:

React + TypeScript + Parcel



You run:

```
npm start
```

and Parcel's dev server starts.

You visit:

http://localhost:1234/



Browser says:

GET /



Who answers?

👉 **Parcel Dev Server.**

It sends something like:

```
<!DOCTYPE html>
<html>
  <body>
    <div id="root"></div>

    <script src="/src/index.tsx"></script>
  </body>
</html>
```

The important thing:

Parcel is serving the HTML.

But look at what's inside the HTML:

```
<div id="root"></div>
```

There is **no React UI** yet.

Then what happens?

Browser receives:

HTML

↓

```
<div id="root"></div>
```



Then browser downloads:

JavaScript bundle



That bundle contains your React application.

Something like:

```
createRoot(document.getElementById("root")).render(  
  <App />  
)
```

Then:

Browser

↓

downloads JS

↓

executes React

↓

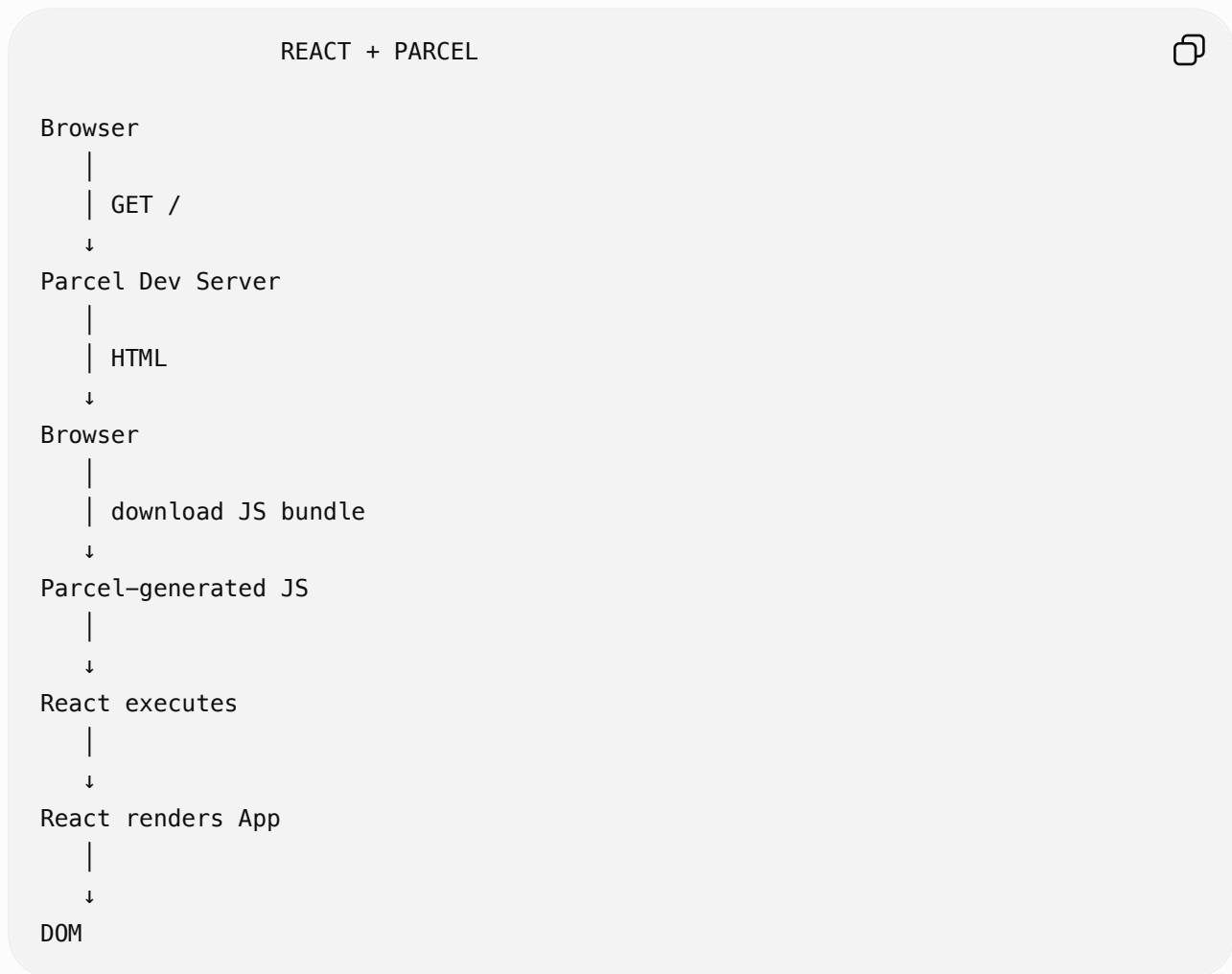
React renders <App />

↓

DOM gets populated



So React + Parcel is approximately:



The key:

Parcel sends the initial HTML.

React generates the actual UI in the browser.



Now Next.js

Now let's take:

Next.js + React + TypeScript

You run:

```
npm run dev
```

and visit:

http://localhost:3000/products



Browser:

GET /products



Who answers?

👉 **Next.js Dev Server.**

Not Turbopack.

Turbopack is involved in processing/building your application assets, but the **Next.js server handles the HTTP request.**

But here's the BIG difference

In Next.js, the server can execute your React Server Components.

Suppose:

```
// app/products/page.tsx

export default async function ProductsPage() {
  const products = await getProducts()

  return (
    <div>
      <h1>Products</h1>

      {products.map(product => (
        <div key={product.id}>
          {product.name}
        </div>
      ))}
    </div>
  )
}
```

Browser:

GET /products



Next.js server:

```
GET /products
↓
find app/products/page.tsx
↓
execute ProductsPage()
↓
getProducts()
↓
React Server Components render
↓
generate result
```



Then Next.js can send HTML like:

```
<div>
  <h1>Products</h1>

  <div>iPhone</div>
  <div>MacBook</div>
  <div>AirPods</div>
</div>
```

So:

```
Browser
↓
GET /products
↓
Next.js Server
↓
React Server Components execute
↓
HTML generated
↓
Browser
```



The key:

Next.js server sends HTML that already contains the rendered UI.

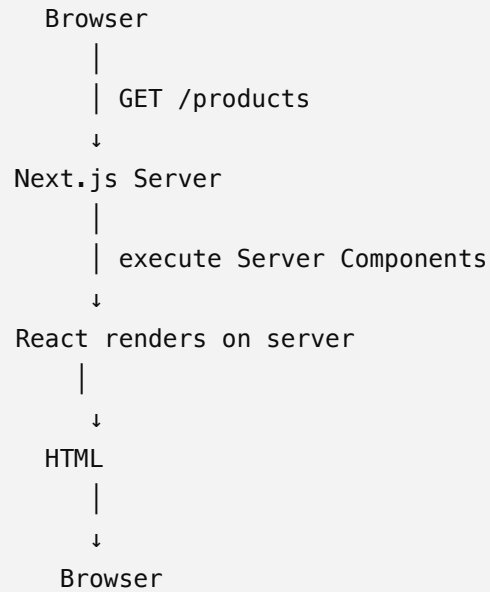
 **THIS is the difference you're looking for**

Put them directly next to each other:

REACT + PARCEL



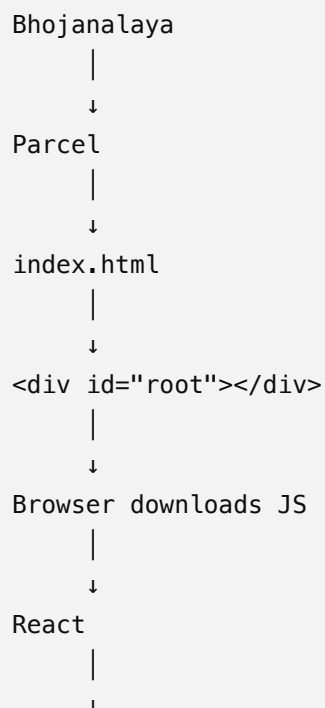
NEXT.JS



This is the **fundamental difference**.

Let's use your Bhojanalaya app example 🍛

Your React + Parcel app is roughly:



```
<App />
  |
  ↓
Bhojanalaya UI
```

So if you inspect the initial HTML source, you might basically see:

```
<div id="root"></div>
```

and scripts.

The actual restaurant UI is created by React **after JavaScript runs**.

Next.js Bhojanalaya equivalent

Imagine we rebuild it using:

Next.js App Router



and:

app/page.tsx



contains:

```
export default function Home() {
  return (
    <>
      <h1>Gopi's Bhojanalaya</h1>
      <RestaurantMenu />
    </>
  )
}
```

Browser:

GET /



Next.js:

```
route /  
  ↓  
app/page.tsx  
  ↓  
execute Server Components  
  ↓  
React server rendering  
  ↓  
HTML
```



Browser receives something conceptually like:

```
<h1>Gopi's Bhojanalaya</h1>  
  
<div>  
  Idli  
</div>  
  
<div>  
  Dosa  
</div>  
  
<div>  
  Pongal  
</div>
```

So the browser **already has visible HTML** before React's client-side JavaScript takes over the interactive parts.

Now the question: Where is Turbopack?

This is where you were getting tangled.

Think of Turbopack as being **behind the scenes preparing your application code**.

React + Parcel:

```
Your React code  
  ↓  
Parcel  
  ↓  
JS bundle  
  ↓  
Parcel Dev Server serves assets/HTML
```



Next.js:

Your Next.js code

↓

Turbopack/build tooling

↓

application assets

Meanwhile:

Browser

↓

Next.js Server

↓

route + render

↓

HTML/RSC response



So Turbopack isn't equivalent to the Next.js server.

And this explains why Next.js feels like "React + a server"

React by itself is primarily the UI library:

React

↓

components

↓

render UI



Parcel adds tooling/dev server:

React

+

Parcel

↓

browser application



Next.js gives you the larger application framework:

React

+

Next.js

|

├─ routing

├─ server rendering

└─ Server Components



- └─ data fetching
- └─ caching
- └─ streaming
- └─ layouts
- └─ API/Route Handlers
- └─ client navigation

That's why Next.js can respond to:

GET /products



by actually **running React on the server** and producing the HTML.

One more VERY important distinction

When we say:

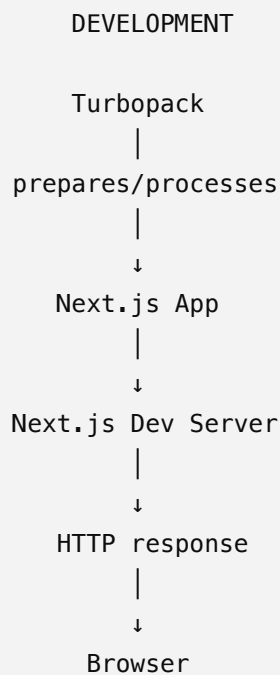
"Next.js sends HTML"

we don't mean:

Turbopack → HTML



Think:



In production:

BUILD



Source code

↓

Next.js build pipeline

↓

production artifacts

REQUEST

Browser

↓

GET /products

↓

Next.js runtime/server

↓

render OR retrieve cached result

↓

HTML/RSC response

↓

Browser



The simplest possible mental model

If you remember **only this**, you're good:

React + Parcel

Parcel server sends a mostly empty HTML shell → browser downloads JS → React creates the UI.

Parcel Server

↓

HTML shell

↓

Browser

↓

React JS

↓

UI



Next.js

Next.js server can run React Server Components → produce the initial HTML → send that rendered HTML to the browser → Client Component JS later provides interactivity.

Next.js Server



Server Components execute



HTML with rendered UI



Browser



Client Component JS



Interactivity



And **THAT** is the foundation you need before we touch SSG/SSR/ISR.

Because now the next question naturally becomes:

"Okay, if Next.js server sends rendered HTML, WHEN does it generate that HTML — during `next build`, when the request comes in, or when the cached version expires?"

And those three answers lead directly into **SSG, SSR, and ISR**. 🔥



okay

Perfect 😊 . We've got the **React + Parcel → Next.js** foundation straight now.

The next piece is:

🔥 Rendering under the hood

We'll take the **same** `/products` page and trace:

1. **SSG** — when HTML is generated at build time
2. **SSR** — when HTML is generated for a request
3. **ISR** — when cached HTML is regenerated
4. **Dynamic rendering in App Router**
5. How `fetch()` and the Data Cache affect rendering
6. How Full Route Cache stores the rendered result
7. How Router Cache comes into play afterward
8. Initial request vs client navigation vs hard refresh
9. What actually executes on the server and what executes in the browser

That'll give you the full end-to-end rendering mental model. 🧠🔥



yes

Absolutely. 🔥 Now let's build the **actual rendering mental model** from the ground up.

The goal isn't just to memorize:

SSG = build time, SSR = request time, ISR = revalidate.

I want you to understand **who executes what, when HTML is created, where it is stored, and what the browser receives.**

Lesson 5.10 — Next.js Rendering Deep Dive

We'll use one example throughout:

/products/123



Imagine:

`</> TypeScript`



```
// app/products/[id]/page.tsx
```

```
export default async function ProductPage({
  params
}) {
  const product = await getProduct(params.id)

  return (
    <>
      <h1>{product.name}</h1>
      <p>{product.price}</p>

      <AddToCart />
    </>
  )
}
```

Where:

```
graph TD; A[ProductPage] --> B[Server Component]; B --> C[AddToCart]; C --> D[Client Component];
```



1. First: What does "rendering" actually mean?

This word causes a LOT of confusion.

In simple terms:

Rendering means turning your React component tree into something that can be used to display/update the UI.

In traditional client-side React:

```
graph TD; A[React Component] --> B[React runs in browser]; B --> C[React creates/updates DOM];
```



In Next.js Server Components:

```
graph TD; A[React Component] --> B[React runs on server]; B --> C[RSC result]; C --> D[HTML can be generated];
```



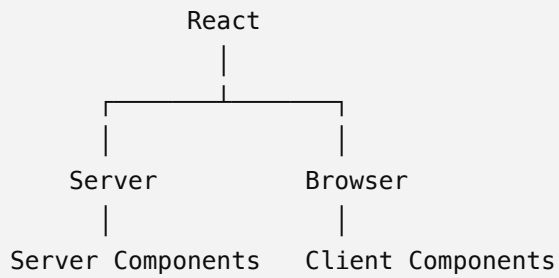
So rendering isn't synonymous with SSR.

Rendering is the process.

SSR/SSG/ISR describe **when/under what circumstances** that server-rendered result is **produced and reused**.

2. There are two major places React can render

Think:



Server

Can execute:

`</> TypeScript`

```
async function ProductPage() {  
  const product = await getProduct()  
  ...  
}
```

Browser

Can execute:

`</> TypeScript`

```
"use client"  
  
function AddToCart() {  
  const [count, setCount] = useState(0)  
  ...  
}
```

This distinction is foundational.

3. Rendering vs hydration

These are different.

Rendering

Produces the initial UI/result.

Hydration

Allows Client Components to become interactive in the browser.

Conceptually:

SERVER



Server Component

↓

render

↓

HTML

BROWSER

HTML

↓

display

Client JS

↓

React

↓

hydrate

↓

interactive

So:

Rendering $\neq$ Hydration



4. Now let's introduce SSG

SSG = **Static Site Generation**.

The key word is:

BUILD TIME

Suppose:

/products/123



is known ahead of time.

During:

```
</> Bash
```

```
npm run build
```

Next.js can render that page.

Conceptually:

BUILD

Source Code

↓

Next.js

↓

ProductPage

↓

getProduct(123)

↓

React Server Rendering

↓

HTML + RSC result

↓

stored as static route output

So the important point:

The HTML isn't being generated because a particular user just requested `/products/123`.

It was generated earlier.

5. Then the user arrives

Hours later:

User → GET `/products/123`

The server can reuse the previously generated result.

Conceptually:

Browser

↓

GET `/products/123`

↓

Next.js

```
↓  
cached static route result  
↓  
HTML  
↓  
Browser
```

That's the basic SSG mental model.

6. React + Parcel comparison

This is where your earlier question becomes useful.

React + Parcel

```
Browser  
↓  
GET /  
↓  
Parcel server  
↓  
HTML shell  
↓  
Browser  
↓  
JS  
↓  
React renders UI
```



Next.js SSG

```
BUILD  
↓  
Next.js renders page  
↓  
HTML generated  
↓  
stored  
  
REQUEST  
↓  
Browser  
↓  
Next.js/CDN  
↓  
stored HTML
```



↓
Browser

So the **React rendering** happened **before** the request.

That's the crucial part.

7. Now SSR

SSR = **Server-Side Rendering**.

Here the rendering happens:

REQUEST TIME

Imagine:

/dashboard



depends on the current logged-in user.

You can't simply generate one HTML file during build because:

User A → different dashboard
User B → different dashboard



So:

```
User A
  ↓
GET /dashboard
  ↓
Next.js Server
  ↓
read request/user information
  ↓
fetch data
  ↓
render
  ↓
HTML
  ↓
User A
```



Then:



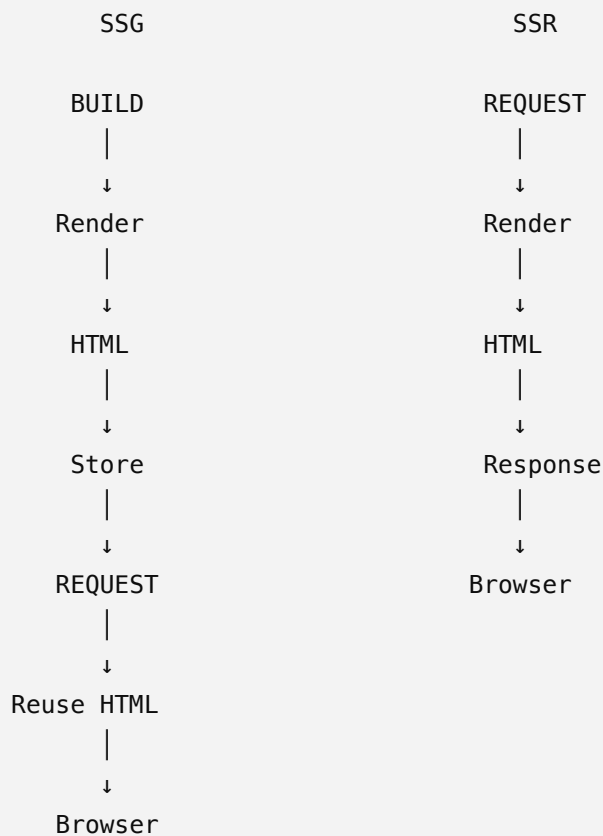
```
graph TD
  UserB[User B] --> GET[GET /dashboard]
  GET --> Nextjs[Next.js Server]
  Nextjs --> render[render for User B]
  render --> HTML[HTML]
```

The rendering occurs **because the request arrived**.

That's SSR's fundamental mental model.

8. SSG vs SSR

Put them side-by-side:



So:

SSG = render earlier.

SSR = render when requested.

9. Now ISR

ISR = **I**ncremental **S**tatic **R**egeneration.

It answers a problem:

"I want mostly-static pages, but I don't want the data to remain static forever."

Imagine:

/products/123



Product price changes occasionally.

You don't want:

every request → render from scratch



but you also don't want:

build once → never update



So ISR gives you:

```
Generate
  ↓
cache
  ↓
serve
  ↓
after revalidation
regenerate
  ↓
new cached result
```



10. Example: ISR every 60 seconds

Conceptually:

```
t = 0
  ↓
generate page
  ↓
cache it
```



Requests:

```
t = 10 → cached version  
t = 20 → cached version  
t = 30 → cached version  
t = 50 → cached version
```



Then revalidation becomes due:

```
t ≈ 60  
↓  
regeneration  
↓  
new version
```



Future requests get the newer result.

11. The classic Pages Router ISR API

You might have seen:

```
</> TypeScript  
  
export async function getStaticProps() {  
  const data = await fetch(...)  
  
  return {  
    props: { data },  
    revalidate: 60  
  }  
}
```



The mental model is:

```
getStaticProps  
↓  
static generation  
↓  
revalidate  
↓  
ISR
```



And:

```
</> TypeScript
```



```
getStaticPaths()
```

tells Next.js which dynamic routes to pre-render.

12. App Router does this differently

Modern App Router doesn't use:

- ✗ `getStaticProps`
- ✗ `getServerSideProps`
- ✗ `getStaticPaths`



Instead, it uses the App Router's Server Component/data/cache model.

For example:

```
</> TypeScript
```



```
const product = await fetch(
  `https://api.example.com/products/${id}`,
  {
    next: {
      revalidate: 60
    }
  }
)
```

Conceptually:

```
fetch
↓
Data Cache
↓
revalidate after 60 sec
```



Then that data participates in rendering/caching of the route.

13. Now the important part: Data Cache

Suppose:

```
</> TypeScript
```



```
const product = await fetch("/api/product/123")
```

There are two separate questions:

Question 1

"Do I already have the fetched data?"

That's:

Data Cache



Question 2

"Do I already have the rendered route?"

That's:

Full Route Cache



Don't confuse these.

14. Full Route Cache

Suppose Next.js renders:

/products/123



and gets:

HTML
+
RSC Payload



The server can cache the rendered route result.

Conceptually:

Full Route Cache



```
/products/123
|
├── HTML
└── RSC Payload
```

So:

```
Data Cache
  ↓
data used for rendering
  ↓
React rendering
  ↓
Full Route Cache
  ↓
rendered route result
```



15. Now connect SSG to Full Route Cache

This is where your previous lessons connect.

For a static route:

```
BUILD
  ↓
fetch data
  ↓
Data Cache
  ↓
render Server Components
  ↓
HTML + RSC Payload
  ↓
Full Route Cache
```



Later:

```
REQUEST
  ↓
Full Route Cache
  ↓
cached route result
  ↓
response
```



So SSG isn't simply:

"HTML file somewhere."

In modern Next.js, the broader architecture involves **rendered route output + caching**.

16. Now connect SSR

For a dynamic route:

```
REQUEST
  ↓
Server
  ↓
dynamic rendering
  ↓
fetch/user data
  ↓
render
  ↓
HTML + RSC Payload
  ↓
response
```



The route isn't simply treated like a permanently static generated page.

That's the SSR mental model.

17. Now connect ISR

ISR becomes:

```
Generate route
  ↓
Full Route Cache
  ↓
serve cached route
  ↓
revalidation
  ↓
regenerate
  ↓
update cached route
```



And the underlying data can also have its own caching/revalidation behavior.

18. Now Router Cache enters the picture

Everything we've discussed so far is primarily **server-side**:

Data Cache
Full Route Cache
Server rendering



Then the browser gets involved.

Suppose the user initially visits:

/products/123



The browser receives the initial response and the App Router can store RSC segments in:

Router Cache



So now:

SERVER



Data Cache
Full Route Cache
Rendering

BROWSER

Router Cache
Client JS
React UI

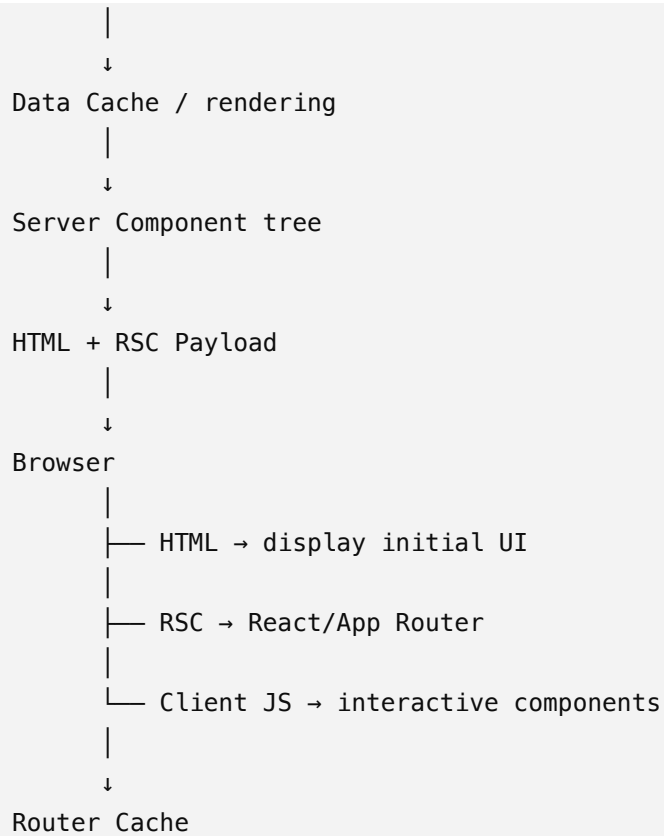
19. Initial navigation

Let's trace it from beginning to end.

USER TYPES URL



↓
Browser
↓ GET /products/123
Next.js Server
↓
Route matching



20. Client navigation

Now user clicks:

`</> TypeScript`

`<Link href="/products/456">`



This time:

Browser



Client Router



Router Cache



If there's a hit:

Router Cache



RSC Payload



React



UI update



If there's a miss:

```
Router Cache MISS
  ↓
request server
  ↓
Next.js
  ↓
RSC Payload
  ↓
Router Cache
  ↓
React
  ↓
UI update
```



No new full document navigation is needed.

21. Hard refresh

Now:

```
Cmd + R
```



The browser requests the document again:

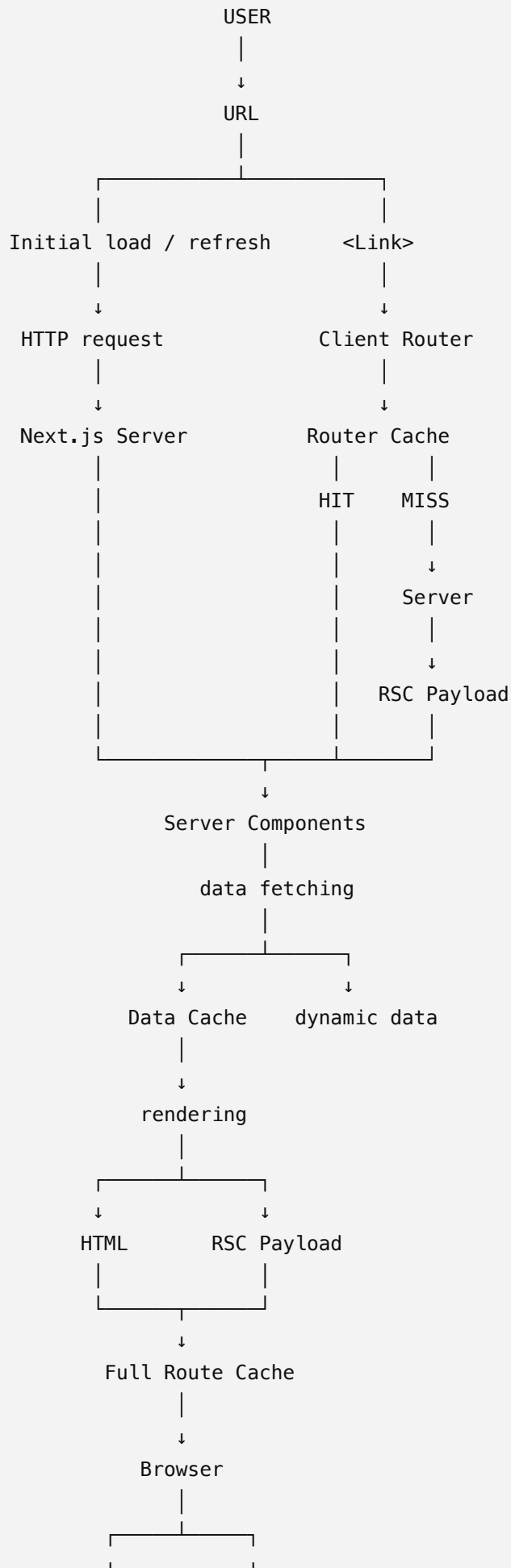
```
Browser
  ↓
GET /products/123
  ↓
Next.js
  ↓
Data Cache / Full Route Cache / rendering
  ↓
HTML + RSC
  ↓
Browser
  ↓
Client JS
  ↓
hydration
```

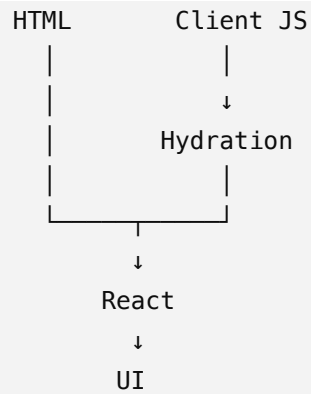


So the Router Cache isn't the thing satisfying the **document request**.

22. This gives us the complete lifecycle

Here's the big picture. 🧠





23. Now the three rendering strategies fit beautifully

SSG

```
BUILD
↓
Render
↓
Full Route Cache
↓
REQUEST
↓
reuse
```



Rendering happens ahead of the request.

SSR

```
REQUEST
↓
Render
↓
Response
```



Rendering happens because the request arrived.

ISR

```
Generate
↓
Cache
```



```
↓  
Requests reuse cache  
↓  
Revalidation  
↓  
Regenerate  
↓  
Update cache
```

Rendering happens initially and again when regeneration is needed.

24. One subtle but crucial thing

Don't think:

```
SSG = Full Route Cache  
SSR = no cache  
ISR = Data Cache
```



✗ That's too simplistic.

Instead:

```
SSG / SSR / ISR  
↓  
rendering strategy / behavior
```



while:

```
Data Cache  
Full Route Cache  
Router Cache  
↓  
caching mechanisms
```



They interact, but they're different concepts.

25. And this is why App Router changed the architecture

Pages Router gave you explicit APIs:

```
getStaticProps  
↓  
SSG
```



```
getServerSideProps
  ↓
SSR

getStaticPaths
  ↓
dynamic SSG
```

App Router instead tries to let the **component/data/cache model determine the behavior**.

Conceptually:

```
App Router
  ↓
Server Component
  ↓
data requirements
  ↓
cacheability / dynamic behavior
  ↓
rendering
  ↓
RSC + HTML
  ↓
caching
```



That's a much more integrated model.

26. The mental model I want you to keep

If someone asks:

"Where does the data live?"

Data Cache



"Where does the rendered route result live on the server?"

Full Route Cache



"Where does the client router keep previously received RSC segments?"

Router Cache



"Where does the initial visual document come from?"

HTML response



"What makes Client Components interactive?"

Client Component JavaScript + hydration



"When is HTML generated?"

Depends on rendering behavior:

SSG → ahead of request

SSR → request time

ISR → generation + regeneration



"What happens during `<Link>` navigation?"

Client Router

↓

Router Cache

↓

RSC Payload

↓

React UI update



27. One last thing before we go deeper

There is one **very important** modern Next.js nuance we should tackle next:

"What exactly makes a route static or dynamic in the App Router?"

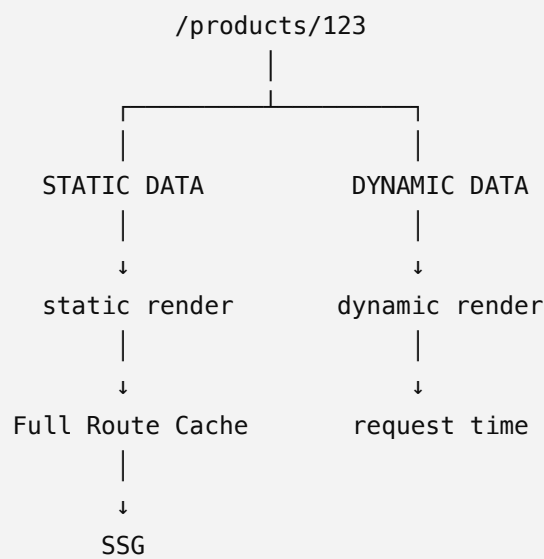
Because then we'll get into:

cookies()
headers()
searchParams
fetch()
cache()
no-store
revalidate



```
dynamic = "force-dynamic"  
dynamic = "force-static"  
generateStaticParams()
```

And we'll trace:



That is the missing bridge between "I understand the caches" and "I actually understand how Next.js decides SSG vs SSR vs ISR." 🔥



Exactly — this phrase is worth unpacking because “**HTTP response**” sounds abstract at first. 😊

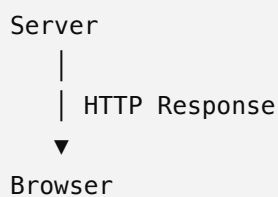
🌐 What is an HTTP response?

When your browser asks a server for something, that's an **HTTP request**.

For example:



The server sends something back. That's the **HTTP response**.



A response can contain things like:



For example:

```
</> http
```



```
HTTP/1.1 200 OK
```

```
Content-Type: text/html
```

```
<html>
```

```
  <body>
```

```
    <h1>Products</h1>
```

```
  </body>
```

```
</html>
```

So when I say:

CDN caches the HTTP response

I mean the CDN can save something like this:

```
GET /products
```

```
↓
```

```
Response:
```

```
200 OK
```

```
<html>Products...</html>
```



and reuse that response for another request.

🔥 Without CDN cache

Suppose you visit:

```
https://example.com/products
```



The flow is:

```
Browser
```

```
|
```

```
| GET /products
```

```
▼
```

```
CDN
```

```
|
```

```
| cache MISS
```

```
▼
```

```
Next.js
```

```
|
```

```
▼
```

```
HTTP Response
```

```
|
```

```
▼
```



CDN
|
▼
Browser

Next.js might produce:

```
200 0K

<html>
  <h1>Products</h1>
  ...
</html>
```

The CDN can now say:

"This response is cacheable. I'll keep a copy."

So:

```
CDN Cache

/products
  ↓
200 0K
<html>Products...</html>
```

Next user requests `/products`

Now another user asks:

```
GET /products
```

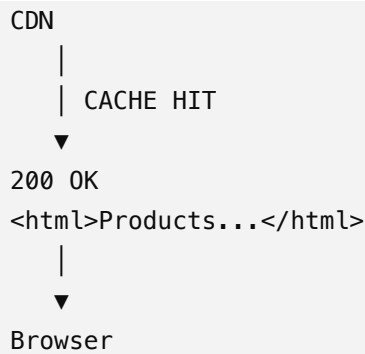
The CDN checks:

```
Do I already have a cached response for /products?
```

YES → Cache HIT 

So:

```
Browser
  |
  | GET /products
  ▼
```



Next.js isn't contacted.

That's what we mean by:

The CDN serves the cached HTTP response.

Important distinction

The CDN isn't necessarily caching some special Next.js object like:

`FullRouteCacheObject`



Instead, from the CDN's perspective, it's dealing with the **HTTP request/response**.

Conceptually:

REQUEST



GET /products



RESPONSE



Status: 200

Headers: ...

Body: <html>...</html>



CDN saves the response



Later:

GET /products



CDN returns previously saved response



And this is different from Next.js Data Cache

This is where your previous questions connect.

Imagine:

Browser



CDN



Next.js



Data Cache



Database/API



Each cache operates at a different level.

Data Cache

Caches **data**:

/products-data



```
[  
  { name: "iPhone" },  
  { name: "MacBook" }  
]
```



Full Route Cache

Caches **rendered route output**:

/products



HTML + RSC Payload



CDN Cache

Caches the HTTP response:

```
GET /products
  ↓
HTTP Response
  ↓
200 + headers + response body
```



Router Cache

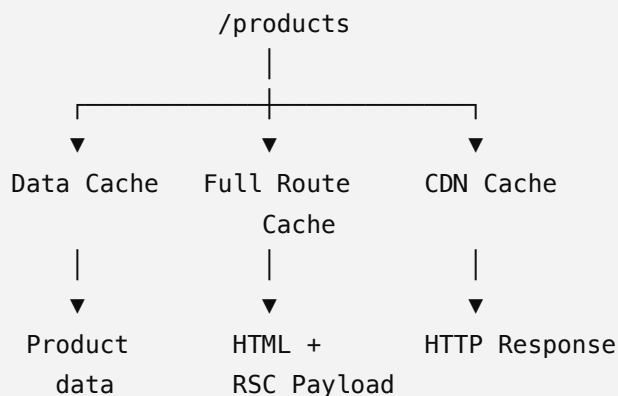
Caches RSC route segments in the browser:

```
/products
  ↓
RSC Payload / segments
```



The simplest mental model

Think about the same `/products` page:



And on the client:

```
Browser
  |
  └ Router Cache
      ↓
    RSC data
```



So when you hear:

"CDN caches HTTP responses"

translate it in your head to:

"The CDN can save the server's entire response to a request and return that saved response directly to future users, without contacting the origin."

That's the key idea. 🗝️

And **this is why CDN cache and Next.js Full Route Cache are related but not the same thing** — both may ultimately involve the rendered page, but they sit at different layers and have different jobs.

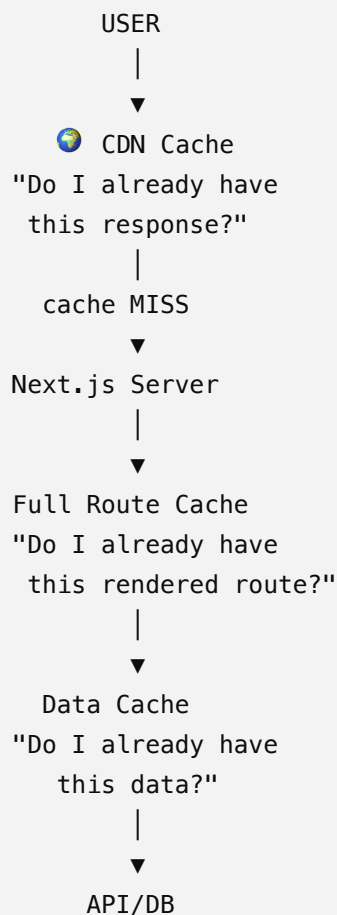


same job right caching..

Yes 😊 — at a very high level, they have the same goal: avoid doing work again.

But they don't have the **same job** because they cache **different things and sit in different places**.

Think of it like this:



The key difference

Cache	What is it avoiding?
Data Cache	Re-fetching data
Full Route Cache	Re-rendering a route
CDN Cache	Going all the way to your origin/server
Router Cache	Re-fetching RSC segments during client navigation

So imagine `/products` .

Without any cache

Browser
↓
CDN
↓
Next.js
↓
API
↓
Database



Lots of work. 🤖

With Next.js caches

Browser
↓
CDN
↓
Next.js
↓
Full Route Cache HIT



Next.js doesn't have to render again.

With CDN cache too

Browser
↓
CDN Cache HIT 🚀
↓
Browser



Next.js isn't even contacted.

🔥 So here's the distinction I want you to remember

Don't think:

"CDN cache and Next.js cache are competing caches."

Think:

"They're multiple layers of caching that prevent different pieces of work at different locations."

It's basically:

CDN

↓

"Can I avoid contacting the server?"

Next.js Full Route Cache

↓

"Can I avoid rendering the page?"

Next.js Data Cache

↓

"Can I avoid fetching the data?"

Router Cache

↓

"Can I avoid requesting RSC segments during navigation?"

So yes, same broad purpose — but different optimization points. 🎯

And this layered-cache idea is going to make **SSG vs SSR vs ISR** much easier to understand, because those determine *whether and where* a route can be cached.



ok

👍 Perfect.

Next logical piece is:

How Next.js decides: "This route is static or dynamic?"

That connects everything we've covered:

```
fetch()
cookies()
headers()
searchParams
no-store
revalidate
generateStaticParams()
    ↓
Static or Dynamic?
    ↓
SSG / SSR / ISR
    ↓
Full Route Cache
    ↓
CDN Cache
    ↓
Router Cache
```



That's the piece we should tackle next. 🧠🔥



ChatGPT can make mistakes. Check important info.